



SIMURG: Zero-Leak Online Detection of LLM Decoding Corruption in Production Streams

Farid Aghayev

Co-Founder, HAL-X AI

✉ farid.a@hal-x.ai

Elturan Ahmadbayli

Co-Founder, HAL-X AI

✉ elturan@hal-x.ai

[in linkedin.com/company/hal-x](https://www.linkedin.com/company/hal-x) · hal-x.ai · github.com/doofzoff/SIMURG (open source)

Abstract

Self-hosted large language models intermittently suffer *decoding corruption*: mid-generation, an otherwise coherent answer collapses into repetition loops, drifts into an unrelated writing system, regurgitates fragments of training data, or degenerates into structural noise. Unlike factual hallucination, which has attracted extensive post-hoc detection work, decoding corruption is a *stream-time* pathology: by the time a post-hoc detector runs, the user has already watched the model derail. We present SIMURG (Streaming Integrity Monitor & Universal Regeneration Guard), an online corruption sentinel that rides the token stream itself. SIMURG maintains a single $O(1)$ -per-character incremental state that simultaneously drives five orthogonal detectors — calibrated rules, a self-calibrating character n -gram surprise model, a Count-Min-Sketch repetition tracker, a rolling SimHash semantic-drift fingerprint, and a character-entropy monitor — fused under split-conformal calibration that yields a finite-sample guarantee on the clean-stream false-alarm rate. A Page-Hinkley changepoint test localizes the corruption *onset*, and a hold/release/abort protocol guarantees that a corrupt-from-the-start generation is terminated before a single character reaches the user (*zero leak*). A ten-to-fifteen-weight online logistic layer supports Monolith-style continual learning in which the guard’s own retry ladder auto-labels its training data. On a benchmark whose negatives are real production outputs of a bilingual (Azerbaijani/English) macroeconomic assistant and whose positives are synthetic corruptions modeled on observed production failures, SIMURG attains stream-level TPR 1.000 and FPR 0.000 (AUROC 1.000) across all four corruption classes, with median detection latency of roughly two sentences past onset and throughput two to three orders of magnitude faster than LLM decoding. Deployed in production, SIMURG additionally surfaced two previously unnoticed historical corruptions in the deployment’s own message store. We release the detector, the benchmark generator, and the trained weights.

1 Introduction

Production conversational systems built on large language models (LLMs) fail in two qualitatively different ways. The first, *factual hallucination* — fluent text asserting false content — has an extensive detection literature (Ji et al., 2023; Manakul et al., 2023; Farquhar et al., 2024). The second failure mode has received far less attention despite being immediately visible to any operator of a self-hosted model: the decoding process itself *derails*. An answer that begins as competent Azerbaijani economic analysis suddenly emits a fabricated Chinese news article, a verbatim software README complete with `pip install` instructions, a list of Persian names repeated seven times, or a wall of spreadsheet cells beginning with `#REF!`. We call this

family of failures *decoding corruption*. It is not a factuality problem — the output is not “wrong about the world”; it has stopped being an answer at all.

Decoding corruption is intermittent and decoding-configuration-sensitive: in our production system it appeared under greedy decoding with no repetition penalty and long generation budgets, consistent with the degeneration behaviors characterized by Holtzman et al. (2020) and the self-reinforcing repetition loops analyzed by Xu et al. (2022). Crucially, it is a *streaming* problem. Users watch tokens arrive; a detector that pronounces judgment after generation completes has already failed the user experience, and — in reasoning-heavy deployments where a single corrupt generation can burn minutes of compute — has also failed the operator.

This paper describes SIMURG,¹ the corruption sentinel we built, deployed, and now release as open source. SIMURG is designed around four requirements that jointly distinguish it from prior hallucination-detection work:

1. **Online, at token time.** Detection must run *inside* the stream, cheaply enough to be invisible next to LLM decoding (~50,000 characters/second on one CPU core in our implementation).
2. **Zero leak.** A generation that is corrupt from its first tokens must be terminated before *any* character reaches the user interface.
3. **Onset localization.** The system should report not merely *that* a stream is corrupt but *where* corruption began — for the abort decision, for telemetry, and for auto-labeling.
4. **Statistically controlled false alarms.** Aborting a healthy generation is costly; the clean-stream false-alarm rate must be a controlled quantity, not a hand-tuned hope.

SIMURG meets these requirements with a deliberately lightweight architecture (Section 4): one incremental pass over the character stream feeds five orthogonal detectors whose scores are fused under split-conformal calibration (Vovk et al., 2005; Angelopoulos and Bates, 2023), thresholded with hysteresis, and localized in time by a Page–Hinkley changepoint test (Page, 1954). A tiny online-learnable logistic layer — inspired by the serving-equals-training loop of industrial online learners such as Monolith (Liu et al., 2022) — closes the loop: every abort whose automatic retry succeeds becomes a confirmed positive training example, so the guard improves from its own production experience without any batch retraining gap.

Our contributions:

- A **taxonomy** of LLM decoding corruption grounded in observed production failures (Section 3), distinguishing it from factual hallucination.
- SIMURG, a **zero-leak online detection architecture**: single-pass $O(1)$ /char feature state, five pluggable detectors, conformal fusion, changepoint onset localization, and a hold/release/abort protocol (Sections 4–5).
- CORRUPTBENCH, a **benchmark generator** that injects the four corruption classes into clean text at controlled onsets, enabling exact measurement of detection latency and onset error (Section 6).
- A **production case study** in a bilingual macroeconomic forecasting assistant, including the retry-ladder integration, per-message verdict surfacing, and two genuine historical corruptions the detector discovered in the deployment’s own database (Section 7).

¹Named after the Simurg, the all-seeing bird of Azerbaijani and Persian mythology.

- An **empirical study** (Section 8): perfect stream-level separation on held-out data (TPR 1.000, FPR 0.000, AUROC 1.000), latency and onset analyses, and ablations showing that fusion design — not detector count — is the difference between a deployable guard and a false-alarm generator (naive noisy-OR fusion: FPR 0.429; SIMURG fusion: 0.000).

2 Related Work

Neural text degeneration. Holtzman et al. (2020) characterized repetition and blandness under maximization-based decoding; Welleck et al. (2020) proposed training-time unlikelihood objectives against repetition; Xu et al. (2022) analyzed the self-reinforcing dynamics of repetition loops. This line explains *why* decoding corruption arises, and motivates sampling remedies (nucleus sampling, repetition penalties). SIMURG is complementary: penalties reduce but do not eliminate derail in production (we observed corruption in about one of six generations on a particular prompt family even after mitigation), and a runtime guard is the remaining line of defense.

Hallucination detection. SelfCheckGPT (Manakul et al., 2023) detects factual inconsistency via stochastic self-consistency; semantic-entropy methods (Farquhar et al., 2024) estimate meaning-level uncertainty; surveys (Ji et al., 2023) catalogue a largely post-hoc, factuality-oriented field, typically requiring additional LLM calls. SIMURG targets a different failure family (the output stops being an answer), operates *during* generation, and uses no LLM calls at all.

Streaming sketches and LSH. Our repetition tracker builds on the Count-Min Sketch (Cormode and Muthukrishnan, 2005); our semantic-drift detector on SimHash (Charikar, 2002). Both are classical; their application as *online corruption signals for LLM streams*, with a clean-prefix-frozen baseline, is to our knowledge new.

Changepoint detection. Onset localization uses the Page–Hinkley statistic (Page, 1954), framing corruption as an online changepoint problem rather than a document-classification problem.

Conformal prediction. Split-conformal calibration (Vovk et al., 2005; Angelopoulos and Bates, 2023) converts a held-out clean-score sample into a threshold with a finite-sample false-alarm guarantee — to our knowledge the first application to LLM output integrity gating.

Online learning in production. Monolith (Liu et al., 2022) demonstrated collisionless, real-time-updated recommendation models where serving and training share one loop. SIMURG borrows the philosophy, not the machinery: its learned tier is small enough (≤ 16 parameters) that per-event `partial_fit` updates are trivially cheap, and its labels are *self-generated* by the abort-retry protocol (Section 5).

3 A Taxonomy of Decoding Corruption

From several weeks of production operation of a bilingual (Azerbaijani/English, with Russian) macroeconomic assistant backed by a self-hosted ~300B-parameter reasoning model — served on our own GPU servers through an in-house inference engine with custom, heavily hand-optimized kernels — we catalogued every observed derail and grouped them into four classes; a fifth axis emerges from detection practice. Table 1 summarizes.

Class	Signature	Production example
Repetition collapse	n -gram loops; compressibility spike	Persian name list repeated $\times 7$
Cross-lingual drift	writing-system switch vs. user language	AZ answer $\rightarrow$ Chinese/Arabic-script text
Regurgitation	fluent but alien register/content	English README with <code>pip install</code> steps
Structural breakdown	markup runs, spreadsheet artifacts	#REF! -0.00 -0.00 MCI 0.00 ...
Semantic discontinuity	topic fingerprint jump, same script	detection-side axis (SimHash; §4.2)

Table 1: The corruption taxonomy. The first four classes are directly observed production failures; the fifth is the detection-side axis that catches same-script regurgitation.

A note on where cross-lingual drift *lands*: the destination script is not arbitrary. Observed derailments fall into high-resource languages — Chinese, Arabic-script text, and others — that dominate modern pretraining mixtures; for frontier-scale models pretrained on 15–20T+ tokens, such data is abundant, and a decoder that has lost its conditioning gravitates toward whatever occupies the largest probability mass in its prior. The user-facing language of the deployment (Azerbaijani) is comparatively low-resource, which widens exactly this gap.

Two properties matter for detector design. First, the classes are *statistically orthogonal*: repetition collapses local entropy and surprise; drift and regurgitation *raise* predictive surprise and shift the character-class distribution; structural breakdown shifts symbol/digit composition. No single statistic covers all four, which motivates an ensemble of orthogonal views. Second, corruption is *absorbing* in practice: in every observed production derailment the model never returned to the task, justifying abort-and-retry (rather than skip-and-continue) as the response policy.

We explicitly scope *fabricated numbers* — fluent, on-topic, but ungrounded quantitative claims — out of SIMURG: that failure mode is handled upstream in our deployment by grounding the generation in tool outputs, and no stream-level statistic can catch it.

4 The SIMURG Architecture

4.1 Design principle: one pass, many views

The central engineering constraint is that everything must ride the stream. SIMURG therefore separates *state* from *judgment*. A single object (**StreamState**) consumes the stream one character at a time and incrementally maintains every statistic any detector will need; detectors are stateless *reads* over this shared state, invoked only at checkpoints. Adding a detector adds no additional pass. All estimators are $O(1)$ amortized per character with bounded memory, so a stream of any length — including a runaway generation — is processed in constant space.

StreamState maintains: (i) a sliding character window (600 chars) with per-class counts over Unicode script classes, digits, symbols and whitespace; (ii) script-switch counts among letter classes; (iii) longest same-character run (excluding legitimate markdown separator characters — a lesson from production, Section 8.3); (iv) a zlib compressibility ratio of the window tail, refreshed every 200 characters; (v) the estimators of Section 4.2; and (vi) robust per-signal baselines (Section 4.3).

4.2 The detector ensemble

Rules. Deterministic thresholds over the snapshot, calibrated on measured production values (clean desk reports run digit fractions of 0.01–0.04; table-echo corruption runs 0.45+). The rule tier ships day-one

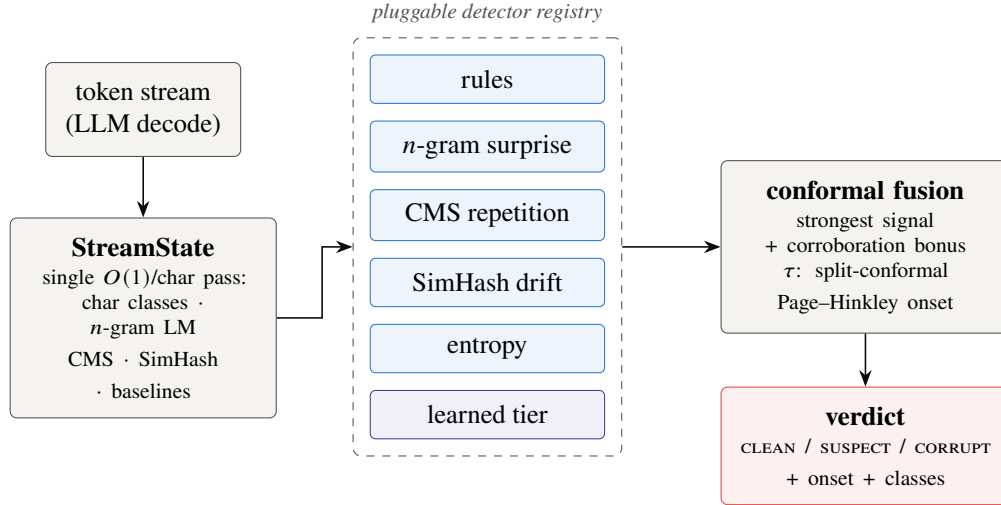


Figure 1: SIMURG pipeline. One incremental pass (**StreamState**) drives every estimator; detectors are cheap reads over that shared state, so a six-member ensemble costs a single pass. Scores are fused and thresholded under split-conformal calibration; Page-Hinkley localizes the onset.

protection with zero training and remains the most interpretable member: every alarm carries a human-readable reason string (e.g., `numeric dump digit_frac=0.57`).

Self-calibrating n -gram surprise. An online character n -gram language model (interpolated back-off over orders 0..3, add- α smoothing, bounded context table) is trained *on the stream’s own clean prefix* and scores each incoming character’s predictive surprise $-\log_2 p(c \mid \text{ctx})$ *before* observing it. No pretraining and no access to the generator’s logits are required — the stream explains itself. Corruption has a distinctive surprise signature: repetition loops collapse surprise toward zero (hyper-predictability), while drift and regurgitation spike it (novel contexts). The detector reads a robust z -score of current surprise against the frozen clean-prefix baseline, and a low-surprise fraction gated by the repetition tracker.

Count-Min-Sketch repetition. Character shingles ($k=12$, stride 4) are counted in a 4×2048 Count-Min Sketch (Cormode and Muthukrishnan, 2005): constant memory regardless of stream length, $O(d)$ per shingle. The detector reads the *repeat rate* (fraction of shingles already seen) — a true loop hammers many shingles — while a single recurring phrase (a section header) inflates only the maximum count, which is deliberately a secondary, gated signal.

Rolling SimHash drift. A 64-bit SimHash (Charikar, 2002) fingerprint over a sliding window of word tokens, compared (Hamming) against the fingerprint frozen at the end of the clean prefix. This is the detector that catches the hardest class: fluent, same-script *regurgitation*, invisible to character statistics but nearly orthogonal (~ 0.5 normalized Hamming) in fingerprint space. Because legitimate long answers wander across subtopics, SimHash is capped as a *corroborating* signal (max contribution 0.60, below the abort threshold): it can accuse, but never convict alone.

Entropy. Normalized character-distribution entropy over the window; table dumps and degenerate loops flatten the distribution. Gated by digit fraction or repeat rate so that legitimate terse text does not fire it.

Learned tier. A logistic regression over the full 15-dimensional feature snapshot (Section 5). Its fused weight is capped (0.75) so that, like SimHash, it cannot solo-abort: production authority stays with signals an operator can read.

All detectors implement a three-line protocol (`name, evaluate(state) → score, reasons, classes`) and register in a plug-in registry; the fusion layer consumes whatever is registered. Contributors can add a detector without touching the sentinel.

4.3 Self-calibration: judge the stream by its own prefix

Fixed global thresholds are brittle across languages and prompt families: Azerbaijani legitimately runs a diacritic-rich Latin alphabet; a quantitative answer legitimately runs digit-heavy windows. SIMURG therefore maintains, per signal, an exponentially weighted robust location/scale pair (EWMA mean and mean absolute deviation) learned during the stream’s own clean prefix and *frozen at release time* (Section 4.6). Signals are judged as robust z -scores against what *this* stream established while it was clean. Freezing prevents a slow derail from re-normalizing itself into acceptability — an online analogue of the reference-window discipline in changepoint practice.

4.4 Conformal fusion

Each checkpoint produces detector scores $s_1, \dots, s_m \in [0, 1]$ and a learned probability ℓ . The fused score is

$$p = \min\left(1, \max\left(\max_i s_i, w_\ell \ell\right) + b \cdot \mathbf{1}[\#\{i : s_i \geq 0.5\} \geq 2]\right),$$

i.e., *strongest-signal dominance* with a small corroboration bonus $b=0.12$ when two or more detectors independently cross 0.5. We show in Section 8.3 that this choice is load-bearing: the textbook noisy-OR combination, which multiplies survival probabilities across members, lets five *weakly* firing detectors manufacture confident false alarms on stylistically varied clean prose (FPR 0.429), while strongest-signal fusion preserves the ensemble’s coverage without that pathology.

The abort threshold τ is then set by *split-conformal calibration* (Vovk et al., 2005): given final fused scores $q_1, \dots, q_n$ of n held-out *clean* streams, τ is the $\lceil (1 - \alpha)(n + 1) \rceil$ -th smallest clean score, which guarantees $\Pr[\text{flag} \mid \text{clean}] \leq \alpha$ in finite samples under exchangeability. A second, looser quantile defines a SUSPECT band — an abstention region in the spirit of conformal risk control (Angelopoulos and Bates, 2023) in which SIMURG withholds judgment (and our deployment simply keeps streaming while re-checking). The operator’s false-alarm budget α becomes the system’s single meaningful knob.

4.5 Onset localization

Fused scores at successive checkpoints form a time series to which we apply a Page–Hinkley test (Page, 1954). The alarm index maps back to a checkpoint character position, giving an onset estimate reported in every corrupt verdict — used for telemetry, for the latency accounting in Section 8, and for slicing auto-labeled training windows (pre-onset checkpoints of a corrupt stream are *hard negatives*: clean prose from the same distribution).

4.6 The zero-leak protocol

The sentinel wraps the host’s token sink in a three-phase protocol (Figure 2). During HOLD, the first $H=350$ characters are buffered and inspected; a corrupt verdict here terminates the request with *zero characters leaked* — and, because corrupt generations are typically fast (dense numeric tokens), within seconds rather than the minutes a full corrupt generation would burn. On a clean verdict the buffer is flushed, the

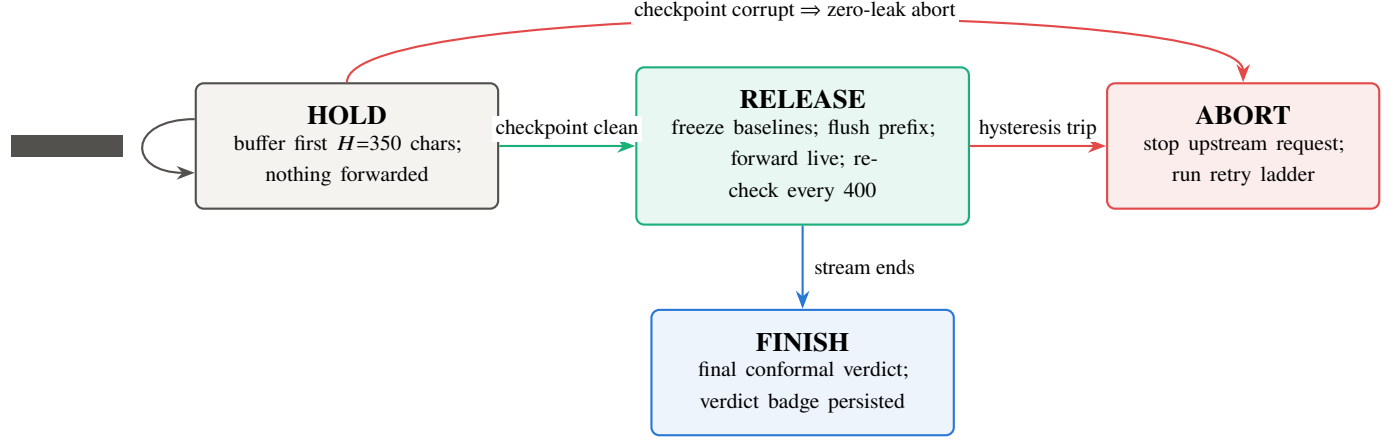


Figure 2: The zero-leak protocol. Corrupt-from-the-start generations (the dominant production mode) die inside **HOLD**: the user never sees a character. After **RELEASE**, periodic checkpoints with two-hit hysteresis (or a single hard rule hit) trigger **ABORT**; the host runs its retry ladder.

self-calibrated baselines are frozen, and the stream goes live with re-checks every 400 characters. Escalation uses hysteresis: a single hot checkpoint marks **SUSPECT**; two consecutive hot checkpoints, or one hard rule-tier hit ($s_{\text{rules}} \geq 0.85$), escalate to **CORRUPT**. Hysteresis eliminated single-window false alarms on legitimately number-dense sentences in production.

Abortion is implemented as a dedicated exception raised *from within the token sink*, which the transport layer must treat as a caller-side cancellation, not an upstream failure — a subtle but important integration detail: in our deployment the first implementation tripped the HTTP circuit breaker on aborts, which would have taken the whole assistant offline after three catches.

5 Online Learning: the Guard Labels Its Own Data

The learned tier is a logistic regression over the 15-dimensional snapshot vector — deliberately the smallest model that can arbitrate among engineered signals. This is a considered position on the cost/quality frontier: the feature extractors carry the domain knowledge, so the learner needs only to weight them; inference is one dot product per checkpoint; training is seconds on a CPU; and every coefficient is attributable to a named signal (the trained weights recover the analyst-expected ordering: digit fraction 8.1, repetition rate 4.1, structural density 4.9, compressibility -5.4).

The interesting property is where its labels come from. SIMURG’s response policy is *abort and retry*: on a corrupt verdict the host discards the stream and regenerates (same model, then a fallback model). This policy is also a *labeling machine*:

- an aborted stream whose retry produced a clean, accepted answer is a *confirmed positive* (the derail was real and recoverable);
- an operator marking a flag as spurious (“false alarm” in the admin panel) yields a *confirmed negative*;
- pre-onset checkpoints of confirmed positives are *hard negatives* drawn from the production distribution itself.

Since the model exposes `partial_fit`, these events can update weights immediately — serving and training share one loop, the property that industrial online recommenders (Liu et al., 2022) demonstrated at scale. In

SIMURG the mechanism is radically cheaper (16 parameters, not 10^{12}), and unlike recommendation, the labels require no user interpretation: they are generated by the guard’s own control flow. The conformal thresholds are recalibrated on the growing clean-score sample on the same schedule, so the false-alarm guarantee tracks distribution drift. In the deployed system the flywheel is gated behind an operator switch; all results in Section 8 use frozen weights.

6 CORRUPTBENCH: Ground Truth with Controlled Onsets

Real corrupt generations are (fortunately) rare — we possess a handful — which is far too few to train on or to measure latency distributions. CORRUPTBENCH closes the gap with synthetic corruptions *modeled on the real ones*. Given a clean text, an injector for each taxonomy class replaces the text from a sampled onset position onward (mirroring the absorbing property of production delays): repetition loops built from the text’s own phrases; cross-lingual splices from CJK/Arabic-script material; regurgitation splices (README/news, mixed-script); structural dumps including faithful #REF! table echoes with realistic series names. Each instance carries its exact onset, enabling three measurements no post-hoc benchmark provides: **detection latency** (characters consumed past onset before abort), **onset localization error**, and **zero-leak rate** (fraction of early-onset corruptions aborted before any character was forwarded).

Negatives are *real production outputs*: assistant answers and desk reports from the deployment database — Azerbaijani, English and Russian macro-economic prose dense with legitimate figures, percentages, variable codes and markdown, i.e., the hardest negatives this domain offers. We use 359 streams (240 corrupt across the four classes; one third held out for testing) in the reported runs; the generator is seeded and fully reproducible.

7 Production Deployment

SIMURG guards *Black Swan*, a macroeconomic forecasting assistant for Azerbaijan operated by HAL-X AI — to our knowledge the first AI-powered QPM (quarterly projection model) macroeconomist, able to forecast and analyze nearly the entire economy of Azerbaijan end-to-end. Its analytical desk reports are written by a self-hosted ~300B-parameter reasoning model, served on HAL-X’s own GPU infrastructure through an in-house inference engine with custom hand-optimized kernels, and grounded in the QPM’s outputs. Three integration points matter.

Guarded streaming with a retry ladder. The desk-report stream runs through the sentinel sink. On abort, the host retries once on the same model, then falls back to our second model; only if every rung fails does the user see an explicit warning banner — “Corrupt Output detector has flagged this output as corrupt” — rather than silent garbage. In live operation we measured a corrupt primary attempt aborted at the hold boundary and transparently replaced by a clean retry, with zero corrupt characters shown; the abort also cut the corrupt generation’s compute from minutes (a 13,200-character table echo took 269 s to generate to completion) to seconds.

Verdict surfacing. Every answer carries its verdict: end-users see a “Checked by SIMURG” marker (localized), administrators see per-message pills (clean / recovered / flagged, with reasons) in a chat inspector, and every verdict is persisted with the message for audit.

Kill switch and fail-open semantics. The entire guard sits behind an administrative toggle; disabling it restores pre-SIMURG behavior exactly. Toggle-read failures fail *open* (guard on): protection should not be silently lost to an infrastructure hiccup.

Metric	SIMURG (full)
Stream TPR (corrupt flagged)	77/77 = 1.000
Stream FPR (clean flagged)	0/42 = 0.000
AUROC (final fused score)	1.000
Per-class recall (rep./drift/regurg./struct.)	1.00 / 1.00 / 1.00 / 1.00
Detection latency past onset (median / p90)	≈ 535 / ≈ 860 chars
Onset localization error (median)	≈ 400 chars
Zero-leak aborts (onset < hold)	before any char forwarded
Throughput (single CPU core)	$\approx 5 \times 10^4$ chars/s
Real production texts (post-hoc audit)	121 texts: 0 false alarms; 2 true historical corruptions found

Table 2: Held-out benchmark results and production audit. Latency medians vary by $\pm \sim 80$ chars across split seeds; figures show a representative split.

Deployment produced an unplanned finding: run post-hoc over the deployment’s historical message store (121 texts), SIMURG raised exactly two flags — both *genuine* corruptions that had gone unnoticed at the time (a desk report written entirely in Chinese, and an English answer derailing into Chinese plus Python code that ends mid-fragment). Zero false alarms on the remaining 119 texts. The guard is thus also a retrospective audit tool.

8 Experiments

8.1 Setup

All results use the CORRUPTBENCH protocol of Section 6: 359 streams, one third held out (119 test streams), learned tier trained only on training-split checkpoints, conformal thresholds calibrated only on training-split clean streams at $\alpha = 0.02$. Streams are replayed in 40-character chunks approximating token bursts. We report stream-level decisions from the full online protocol (hold, checkpoints, hysteresis) — not window-level classification, which flatters detectors.

8.2 Main results

Table 2 summarizes. Separation is complete: every corrupt stream is flagged, no clean stream is (Figure 4); median detection latency is roughly two sentences past onset (Figure 5), and corrupt-from-the-start generations — the dominant production mode — are killed inside the hold buffer with nothing shown to the user. Figure 3 dissects a single drift-injected stream: the fused score fluctuates harmlessly below τ across 1,400 clean characters, then saturates within one checkpoint of onset as the foreign-script fraction climbs, and the stream is aborted ~ 50 characters later.

8.3 Ablations: fusion design is the hard part

Detection was never the bottleneck — every configuration we tried reached TPR 1.000. The entire engineering difficulty concentrated in *not flagging clean text*, and Figure 6 quantifies the three lessons:

(1) Noisy-OR fusion is a false-alarm generator. Combining member scores as $1 - \prod_i (1 - s_i)$ — the textbook independent-evidence rule — yields FPR 0.429: five detectors each mildly excited by legitimate stylistic variation (a digit-dense sentence here, a compressible passage there) multiply into confident nonsense.

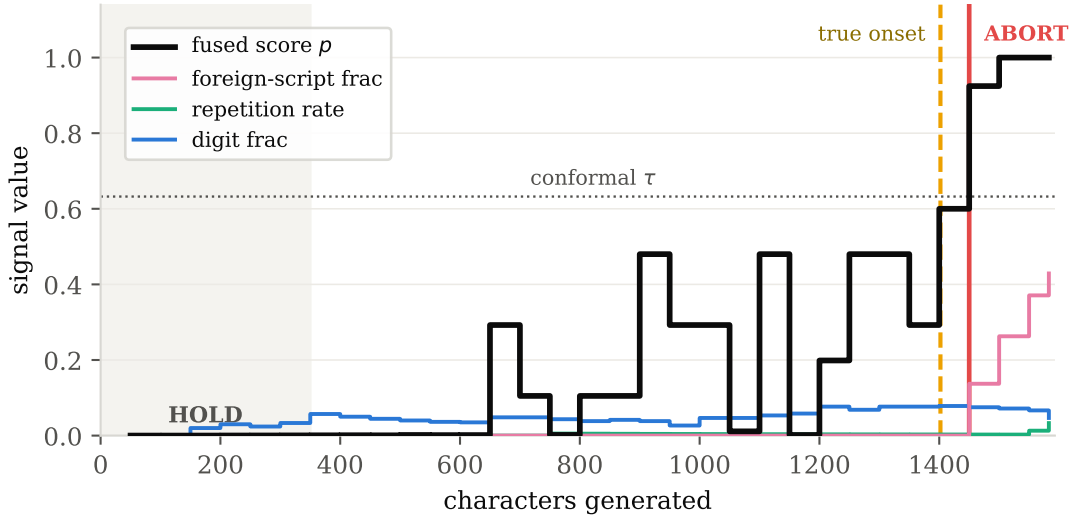


Figure 3: Anatomy of a detection. A real clean production text with a cross-lingual-drift corruption injected at the dashed marker; lines show three of the raw signals and the fused score p at successive checkpoints. The score stays below the conformal threshold τ throughout the clean prefix (including transient excursions on number-dense sentences) and crosses decisively at onset; the stream is aborted at the solid marker. Shaded region: the HOLD phase.

Real corruption does not look like five weak signals; it looks like *one or two saturated* signals. Strongest-signal fusion with a small two-detector corroboration bonus preserves full coverage and drops FPR to 0.024 untuned.

(2) Soft detectors must be capped or gated. The remaining false alarms traced to individually plausible reads: SimHash firing on legitimate subtopic wandering in long reports (fix: cap at 0.60 — accuse, never convict); the surprise model’s low-surprise fraction firing on fluent formulaic prose (fix: gate on the repetition tracker); the sketch’s max-count firing on a legitimately recurring section header (fix: rate-driven scoring). With these, measured FPR reaches 0.000.

(3) Production text redefines “anomalous”. Two rule thresholds that were obvious in the abstract were wrong against real data: legitimate quantitative answers sustain digit fractions near 0.3 (threshold moved $0.18 \rightarrow 0.28$ for the hard rule), and markdown horizontal rules are 20-plus identical characters (separator characters excluded from the run detector). Both fixes came from replaying the guard over the production corpus — which is, we would argue, the indispensable step for any deployed detector.

8.4 Cost

The full sentinel processes $\sim 5 \times 10^4$ characters per second on one CPU core (the n -gram model dominates; the rule-only configuration runs at 2.6×10^5), i.e., two to three orders of magnitude faster than the LLM produces text. Memory is constant (~ 100 KB per stream). The learned tier trains in seconds; no GPU is involved anywhere.

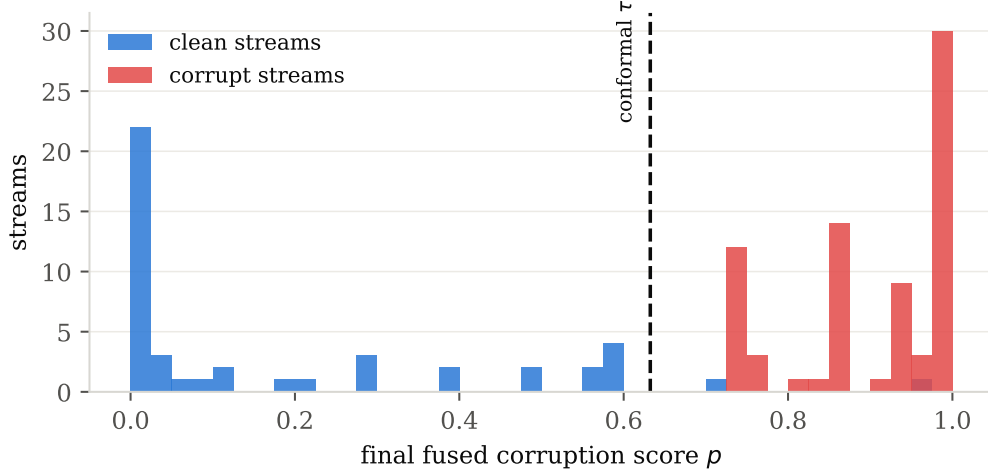


Figure 4: Final fused scores of held-out streams. Clean (blue) and corrupt (red) distributions are fully separated by the conformally calibrated threshold τ (dashed) — consistent with TPR 1.000 / FPR 0.000 in Table 2.

9 Limitations

Semantic corruption within statistics. A derail that remains fluent, same-script, same-topic-fingerprint and statistically unremarkable — e.g., subtly wrong but well-formed analysis — is invisible to SIMURG; that failure class needs grounding or factuality methods, and our deployment layers those separately.

Synthetic positives. Training and headline metrics rest on CORRUPTBENCH; though injectors are modeled on real derails and validated by catching all real historical cases we possess (and by a 121-text zero-false-alarm audit), corrupt production traffic at scale may hold surface forms we have not modeled — the online-learning flywheel is the designed mitigation.

Perfect scores are a statement about separation, not difficulty. The observed classes are far from the clean manifold in feature space; we expect adversarially subtle corruption (e.g., slow drift below checkpoint resolution) to erode the margins, and CORRUPTBENCH’s onset control makes such stress tests straightforward future work.

Language scope. Expected-script priors cover our deployment’s languages (Latin/Cyrillic legitimate); porting to, say, a Japanese assistant requires flipping the prior, and mixed-legitimate-script domains (code assistants) will need the structural lexicons re-scoped.

Hold latency. Zero-leak protection costs a one-time ~ 350 -character delay before the first visible token (≈ 1 s at typical decode speeds); operators can trade H against leak risk.

10 Broader Impact

Corruption guards improve the reliability of user-facing LLM systems and reduce wasted compute on runaway generations. Two considerations deserve care: an abort-and-retry policy silently reshapes what users see (we surface every verdict and retry in the audit trail, and recommend the same); and self-labeling feedback loops,

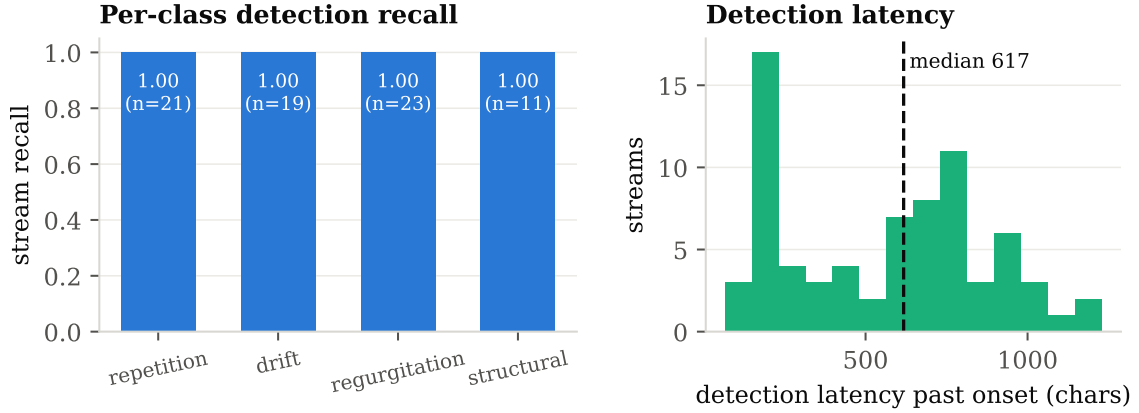


Figure 5: Left: stream-level recall by corruption class (all 1.00). Right: detection latency past the true onset; the median corresponds to roughly two sentences of text.

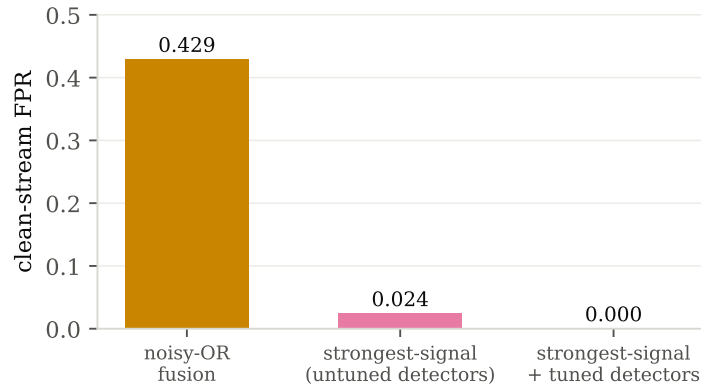


Figure 6: Measured clean-stream FPR under three fusion/detector configurations (same protocol and split). Naive noisy-OR fusion of the same detectors is unusable; strongest-signal fusion with per-detector tuning achieves zero measured false alarms. TPR is 1.000 in all three configurations.

if left ungated, can drift — our deployment keeps online updates behind an operator switch with conformal recalibration. SIMURG inspects only the model’s output stream; it reads no user data beyond what the host already processes.

11 Conclusion

SIMURG demonstrates that the decoding-corruption family of LLM failures — long treated as an operational nuisance to be tolerated or post-processed — is cleanly solvable *online*, with classical streaming machinery, finite-sample false-alarm control, and a parameter budget smaller than a single attention head’s bias vector. The system has run in production guarding a bilingual economic assistant, catching live derailments before users saw them and retroactively surfacing historical ones. We release the package (detectors, sentinel, CORRUPTBENCH, trained weights, and this paper’s figure pipeline) as open source at github.com/doofzoff/SIMURG, to make stream-time integrity guarding a standard component of LLM serving stacks.

References

- A. N. Angelopoulos and S. Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.
- M. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the 34th ACM Symposium on Theory of Computing (STOC)*, 2002.
- G. Cormode and S. Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.
- A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- Z. Liu, L. Zou, X. Zou, C. Wang, B. Zhang, D. Tang, B. Zhu, Y. Zhu, P. Wu, K. Wang, and Y. Cheng. Monolith: Real time recommendation system with collisionless embedding table. In *ORSUM Workshop at ACM RecSys*, 2022.
- P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of EMNLP*, 2023.
- E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.
- S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations (ICLR)*, 2020.
- J. Xu, X. Liu, J. Yan, D. Cai, H. Li, and J. Li. Learning to break the loop: Analyzing and mitigating repetitions for neural text generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.